Advanced ⇒ Complete Programming + Data Structures + Algorithms Syllabus

🔷 SECTION 1: Core Programming Syllabus (0% to 100%)

1️⃣ Foundations of Programming

- What is programming?
- History and types of languages
- Flowcharts, pseudocode
- Paradigms: procedural, OOP, functional, etc.

2️⃣ Variables, Types, and Expressions

- Variables, constants, literals
- Primitive & user-defined types
- Static vs dynamic typing
- Type casting and coercion
- Operators: arithmetic, logical, bitwise

3️⃣ Control Structures

- Conditional statements
- Loops (for, while, do-while)
- Nested, labeled loops
- Jump statements (break, continue)

4 Functions & Modular Programming

- Declaration & definition
- Parameters & return types
- Scope & lifetime
- Recursion (basic → tail → optimization)

5 Object-Oriented Programming

- Classes & objects
- Constructors, destructors
- Encapsulation, inheritance, polymorphism, abstraction
- Composition vs inheritance
- SOLID principles

6 Functional Programming

- Pure functions, immutability
- First-class & higher-order functions
- Closures, currying
- Map, filter, reduce

7 Memory & Performance

- Stack vs heap
- Garbage collection
- Pointers and references (conceptual)
- Profiling & optimization

8 File and Error Handling

- File I/O (text, binary)

- Exception handling
- Logging and debugging

9️⃣ Asynchronous and Concurrent Programming

- Sync vs async
- Callbacks, promises
- Threads, coroutines
- Race conditions, locks, semaphores

🔟 Advanced Programming Concepts

- Generics / templates
- Operator overloading
- Reflection & introspection
- Fluent interfaces, chaining
- Design patterns (Factory, Singleton, Observer, etc.)

🟩 SECTION 2: Data Structures (DS Only)

🧱 Level 1: Linear Data Structures

- Arrays (1D, 2D, dynamic)
- Strings (mutable, immutable)
- Linked Lists: singly, doubly, circular
- Stack: implementation, applications
- Queue: normal, circular, deque

🌳 Level 2: Tree Data Structures

- Binary Trees: structure, traversal
- Binary Search Tree (BST)
- AVL Tree (balancing)
- Heaps: min, max, k-heap
- Trie (prefix tree)
- Segment Trees (point & range queries)
- Fenwick Trees (Binary Indexed Trees)

🌉 Level 3: Graph Data Structures

- Representation: adj list/matrix
- Weighted vs unweighted
- Directed vs undirected
- Disjoint Set Union (Union-Find)

🗂️ Level 4: Hashing

- Hash functions
- Collision handling (chaining, open addressing)
- Custom hash tables
- Hash maps and hash sets

🧩 Level 5: Special Structures

- Priority Queues
- Deques
- Circular buffers
- Tries & Suffix Trees
- Bloom Filters

- Rope data structure
- Persistent Data Structures

🟥 SECTION 3: Algorithms (Only Algorithms)

🔎 Level A1: Searching Algorithms

- Linear Search
- Binary Search (on arrays, answers)
- Ternary Search
- Exponential Search

🧹 Level A2: Sorting Algorithms

- Bubble, Selection, Insertion
- Merge Sort, Quick Sort
- Heap Sort, Shell Sort
- Counting Sort, Radix Sort
- Bucket Sort
- Top-k Sort Techniques

🧠 Level A3: Recursion & Backtracking

- Basic recursion
- Backtracking: N-Queens, Sudoku, permutations
- Optimizations using memoization

📊 Level A4: Dynamic Programming (DP)

- Memoization vs tabulation
- 0/1 Knapsack, Unbounded Knapsack
- Longest Common Subsequence (LCS)
- Longest Increasing Subsequence (LIS)
- Matrix DP, Digit DP, Bitmask DP
- DP on Trees and Graphs

🔥 Level A5: Greedy Algorithms

- Activity Selection
- Fractional Knapsack
- Huffman Coding
- Interval scheduling
- Greedy vs DP comparison

🌀 Level A6: Graph Algorithms

- BFS, DFS
- Shortest Path: Dijkstra, Bellman-Ford, Floyd-Warshall
- MST: Kruskal, Prim
- Topological Sorting
- Union-Find for Cycles
- Tarjan's & Kosaraju's (SCC)

💡 Level A7: String Algorithms

- KMP, Rabin-Karp
- Z-Algorithm
- Trie-based matching
- Suffix Array, Suffix Tree
- Manacher's Algorithm

⚡ Level A8: Advanced Techniques

- Sliding Window, Two Pointers
- Bit Manipulation & Tricks
- Binary Lifting
- Meet-in-the-middle
- Mo's Algorithm (Offline queries)
- Heavy-Light Decomposition (HLD)
- Centroid Decomposition
- Persistent Segment Trees

🎓 Capstone Challenges

- LeetCode 500+ problems
- Write your own library for trees/graphs
- DSA + real-world project (e.g., autocomplete system)
- Participate in Codeforces/AtCoder contests

✅ Summary View

Category	Coverage
Programming	Beginner → Expert
Data Structures	Arrays → Persistent Trees
Algorithms	Sorting → HLD & Mo's Algo